Exercise 7

Sleep mode with register and interrupts (Microchip Studio) (UNO)

In this task, you will build a program where one button puts Arduino UNO to sleep using the included sleep register and commands, and the other button functions as interrupt that wakes up the Arduino UNO.

Build the circuit using these components:

- Arduino UNO
- 2 x buttons
- 2 x 10 kΩ resistors
- LCD display
- 1 x Potentiometer
- 1 x 220 Ω resistor
- Wires

Make the connection of the LCD pins as below:

- D4 – D7 pins are connected to UNO pins 3 – 6
- RS, RW, and EN are connected to UNO pins 9 – 11
- V0 is connected to the potentiometer
- VDD is connected to power
- VSS is connected to ground
- K is connected to ground
- A is connected to 220 Ω resistor, and power

First button for sleep mode: Connect one leg to ground via a 10 kΩ resistor, and the same leg also to pin 7 of Arduino UNO, while the other leg is connected to power.
Second button for wake up: Connect one leg to ground via a 10 kΩ resistor, and the same leg also to pin 2 of Arduino UNO, while the other leg is connected to power.

<u>Start to code:</u>

- Remember to *#define* the clock speed with `F_CPU 16000000UL` and constant delay with `DELAY 100`
- Remember to *#include <avr/sleep. h>, <avr/interrupt.h>,<util/delay.h>, <stdlib.h>, and "lcd.h"*
- Create a variable for counting the time that has passed since the program started.
  ```
  float time_from_start = 0; //to count time from start
  ```
- Create a variable for counting how many times the sleeping mode has been interrupted.
  ```
  volatile float counter = 0; //to count how many interrupt that has been done.
  ```
- Implement the interrupt function with the counter that increase the value by one.
  ```
  ISR(INT0_vect){
     counter =counter + 1; //means increase the counter value by one
  }
  ```

<u>In the main void</u>, include these:

1. Set up the port pin for digital pin 2 and 7
   ```
   DDRD &= ~(1 << PD2) & ~(1 << PD7);
   ```
2. Set up the external interrupt mode using EICRA register. See datasheet p. 71.
   ```
   EICRA |= (1 << ISC01);
   ```
3. Use the falling edge to create the interrupt. Enable the interrupt INT0 which is located in the pin 2 using EIMSK register. See datasheet p.71.
   ```
   EIMSK |= (1 << INT0);
   ```
4. Set the sleep mode using SMCR register. See datasheet p. 44.
   ```
   SMCR |= (1 << SM1);
   ```

5. Enable the global interrupts.
```
sei();
```
6. Initialize the LCD. You also can add the clear screen command.
```
lcd_init(LCD_DISP_ON);
```

In the while loop:

1. Convert the interrupt value, which is in float type, to a string using dtostrf.
```
char count[16];
dtostrf(counter, 3, 0, count); //dtostrf(float_value, min_width, num_digits_after_decimal, where_to_store_string)
```
2. Print the 'interrupt counter' in the LCD using lcd_puts.
3. Using if-structure to make the situation; when you press the "sleep"-button, it enables the sleep and activate the CPU to sleep. When "wake up"-button is pressed, the sleep is disabled.
```
if((PIND & (1 << PD7))!= 0) {
SMCR |= (1 << SE); //enable sleep, see datasheet p.44
sleep_cpu();       //activate CPU to sleep
SMCR &= ~(1<<SE);  //disable sleep, see datasheet p.44
}
else {
```
4. Convert the time from float type to string.
```
time_from_start = time_from_start + 1; //counting the time that has elapsed from the starting of program
char time_elapsed[16];
dtostrf(time_from_start/((float)1000/DELAY), 3, 2, time_elapsed); //change the float to character array and converts time units into seconds
```
5. Print the time in the LCD.